Micro-VLA: Scaling In-Context Robotic Manipulation via Observation-Only Videos *

Adrian Boguszewski, Bartosz Czechowski

June 23, 2026

Opening Remarks

Throughout the report, the model that we are discussing is not a VLA. Initially we experimented with a pretrained, 500M parameter VLA - *SmolVLA*, however we quickly realized that it wasn't computationally feasible for us. One epoch of fine-tuning the action head (not even the VLM backbone) took around an hour on a Google Colab T4 GPU, and yielded around 10% success rate on the simplest task we were interested in. We decided to train an order of magnitude smaller decision transformer from scratch (whose architecture will be disclosed in a later section).

1 The Environments

We decided to use *robosuite*, which its authors describe as "a simulation framework powered by the MuJoCo physics engine for robot learning (...) (that) also offers a suite of benchmark environments for reproducible research". The decision was motivated by its easily configurable environments with an option to swap out robot arms to any of the available models.

1.1 Tasks

We chose a subset of three tasks, based on the following criteria: they have an obvious difficulty ordering, they can be seen as a natural extension of one another, and they have accompanying dataset (with an asterisk described in a later section).

1.1.1 Block Lifting

A cube is placed on the tabletop in front of a single robot arm. The robot arm must lift the cube above a certain height. The cube's location is randomized at

*We dropped the "via Observation-Only Videos", as discussed with our lab teacher.

the beginning of each episode.

Figure 1: *Block Lifting* task demonstration

1.1.2 Block Stacking

Two cubes (red and green) are placed on the tabletop in front of a single robot arm. The robot must place the red cube on top of the green cube. The cubes' locations are randomized at the beginning of each episode.

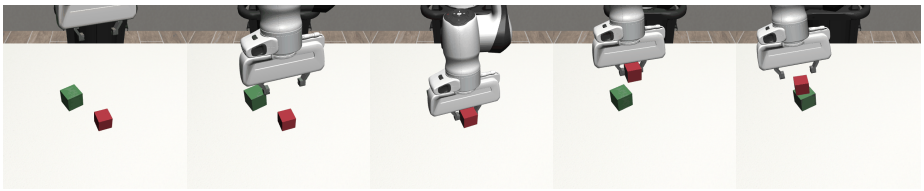


Figure 2: *Block Stacking* task demonstration

1.1.3 Nut Assembly (one square nut variant)

Two colored pegs (one square and one round) are mounted on the tabletop, and a square nut is placed on the table in front of a single robot arm. The robot must fit the square nut onto the square peg. The nut's location is randomized at the beginning of each episode, while the pegs are fixed.

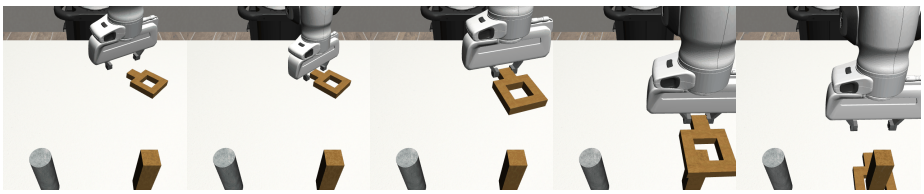


Figure 3: *Nut Assembly* task demonstration

1.2 Robot Arms

Throughout the project we use two robot arms: Panda and IIWA. The first one was chosen, because it was present in the datasets that we had access to

(described later). The second one was selected for its similarity to the Panda - 6 degrees of freedom with similar kinematics and a single-degree-of-freedom gripper. Two distinct arms were necessary to evaluate cross-embodiment generalization.

Figure 4: Panda arm

Figure 5: IIWA arm

2 Training Data

As our training data we mainly used demonstrations sourced from *robomimic*. In its totality, *robomimic* is a more general framework, but we used it solely as a source of human demonstrations and dataset processing scripts.

From *robomimic* we took proficient human and multi-human (divided into "better", "okay" and "worse" operators) demonstrations for block lifting and nut assembly tasks (using the Panda arm), as well as the machine generated trajectories for block lifting (again, using the Panda arm).

What we couldn't find in *robomimic's* archive, we collected ourselves - block stacking demonstrations (using Panda and IIWA arms), and block lifting demonstrations for the IIWA arm.

We also had to process those datasets to prepare the camera observations, rewards, and feature extractor embeddings of the camera observations for training efficiency (the feature extractor is described in the following section). The original datasets only contained raw trajectories (action, joint and environment state sequences).

All of our training data is publicly available on *HuggingFace* with a comprehensive dataset card that goes into more technical details.

3 Model Architecture

We used a rather standard Decision Transformer architecture - with some implementation details described below. The whole implementation of this project is available on GitHub.

3.1 Rewards

We made using rewards optional - as when learning from expert trajectories they don't carry much information. But if one decides to use rewards (possibly due to introducing sub-optimal trajectories in the dataset), it's configurable whether to use the dense rewards from the dataset (which grow as the model gets closer to the goal) or to use time based rewards (where we give a reward of -1 at every step, and +100 on success). Then these rewards are used in the return-to-go fashion from the original Decision Transformer paper.

Using dense rewards is sub-optimal as it can be misleading - a model that takes longer to pick up the cube will get a higher total episode return than a model that picks it up faster.

3.2 Observations

As input the model takes a proprioception vector (end-effector position, end-effector rotation, gripper width) and two precomputed Dino3 ViT-Small embeddings of the images from two cameras (eye in hand and robotview). It can be configured whether these should be concatenated into one vector which will be embedded into the latent space of the model, or whether they should be embedded separately and then attended to separately by the model.

We have found that fusing the observations into one gives better results than attending to them separately.

3.3 In-context learning

We also implemented in-context learning - where we also provide the model with a successful trajectory of the task (during training and evaluation).

3.4 Positional Embeddings

We have decided to use learnable positional embeddings as in the original Decision Transformer paper. When using in-context learning we have decided to reset the timestep counter to zero.

Using sinusoidal positional embeddings and not resetting the timestep counters were considered but not evaluated.

3.5 Training

We have implemented a standard training loop with MSE loss. We have enabled training on multiple datasets with configurable weighed sampling.

3.6 Initial Experiment

We performed an initial experiment to test whether this architecture was capable of learning the simplest of our tasks - block lifting. The experiment was successful (see Figure 6), however we came across a weird phenomenon (also

described in the original *robomimic* paper). The validation loss didn't correlate with model's performance inside the environment. This meant that for every experiment we had to perform validation rollouts inside the environment to accurately grasp model progress, and that it is worth to train the model much longer than we initially thought.

Another important thing to note is that the training is really unstable and the evaluation carries great variance.

Figure 6: Training and validation losses, as well as success rates plotted against the training epochs on the block lifting task.

4 Task Generalization

We performed a large number of experiments that tested generalization between tasks.

4.1 Color Change

A model trained on the block lifting task with a red cube exhibited equal zero-shot performance on cubes of different colors (see Figure 7). However, after inspecting the actual produced trajectories (also failures) we noticed that unfortunately it's not true generalization, but instead overfitting to block position (a recurring problem in this project).

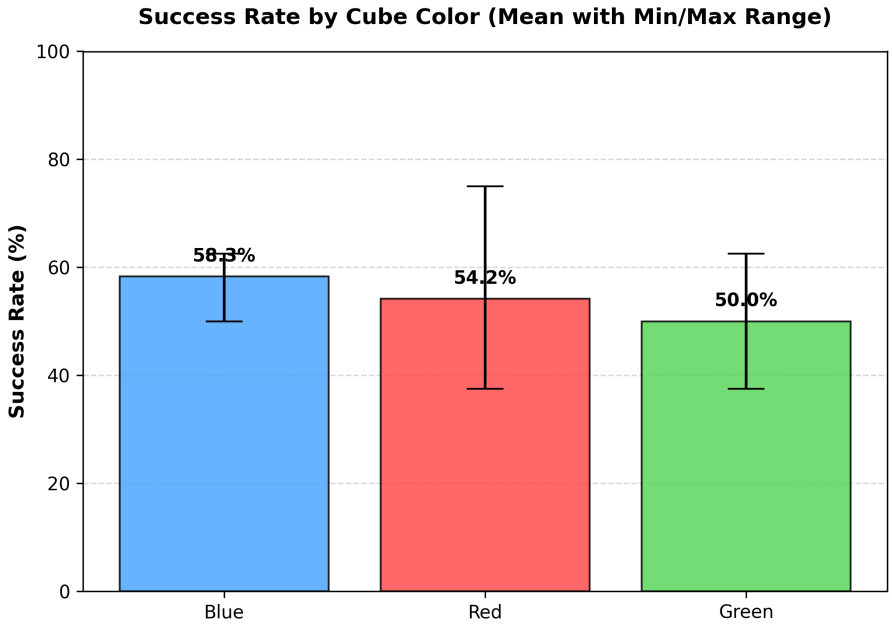


Figure 7: Model performance on different cube colors.

4.2 Training with In-Context Prompts

A model trained with in-context prompts didn't achieve better one-shot performance than a baseline zero-shot model on the trained task (see Figure 8 and remember that the best version of the initial model scored $\sim 90\%$).

4.3 Scaling Up Training with In-Context Prompts

Introducing time-based rewards (-1 for each step and +100 for success) and increasing the size of training data degraded performance on the trained task. The dataset was extended with demonstrations of lower quality - multi-human (MH), so datasets collected by multiple human operators of varying proficiency level, and machine generated ones (MG). Previously the models were trained on proficient human (PH) demonstrations. For the exact results see Figures 9, 10, 11. Adding demonstrations of the nut assembly task also failed to produce any results.

4.4 In-Context Learning

All previously described models trained with in-context prompts, were evaluated on the block stacking task and achieved $\sim 0\%$ success rate. They were prompted with a block stacking trajectory, but were unable to produce a successful one.

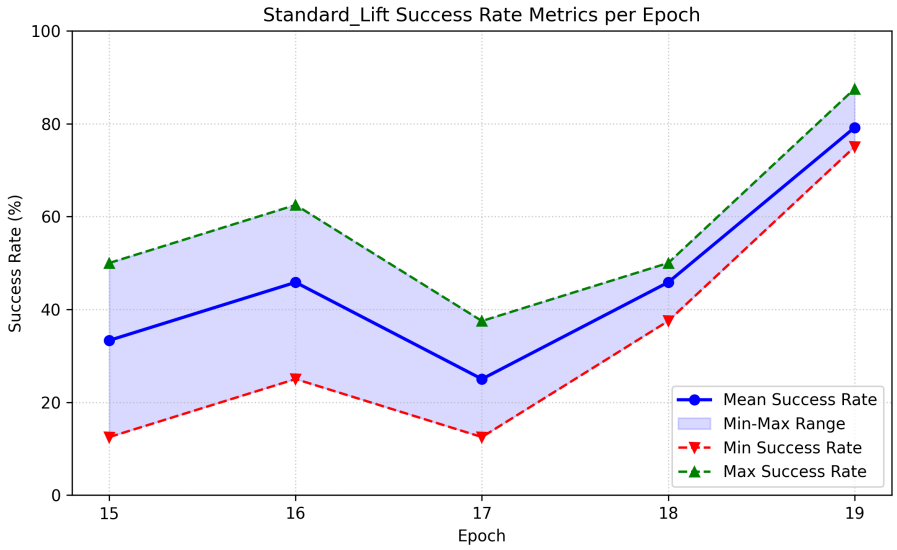


Figure 8: Performance of the model trained with in-context prompts on the block lifting task.

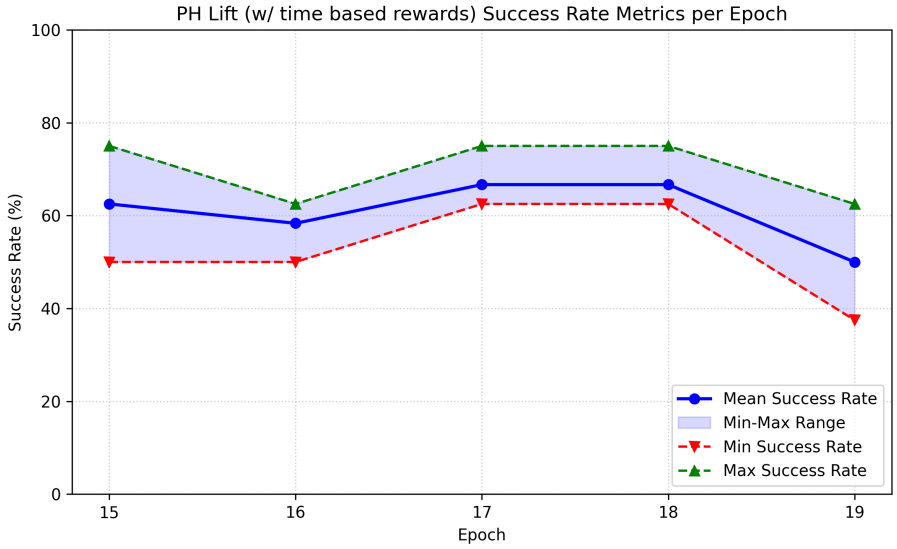


Figure 9: Performance of the model trained with in-context prompts and time based rewards on the block lifting task (PH dataset).

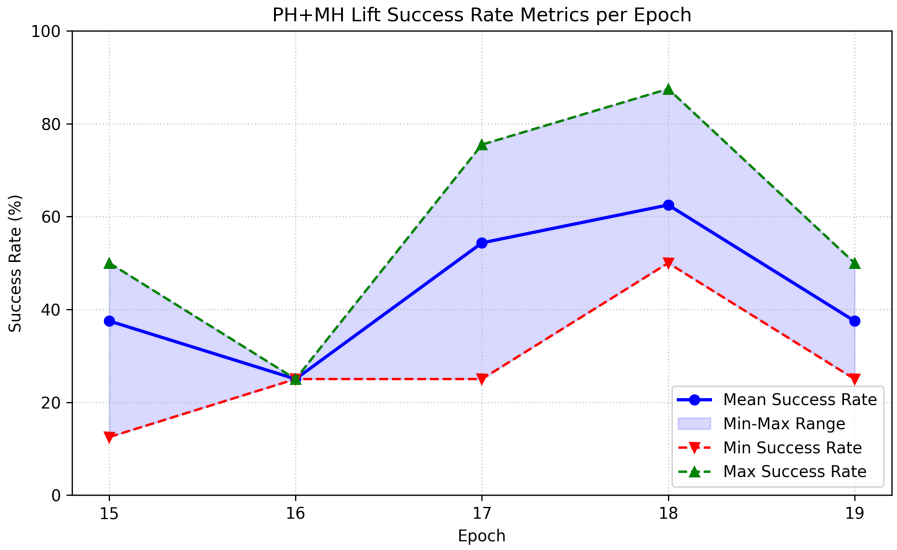


Figure 10: Performance of the model trained with in-context prompts and time based rewards on the block lifting task (PH + MH dataset).

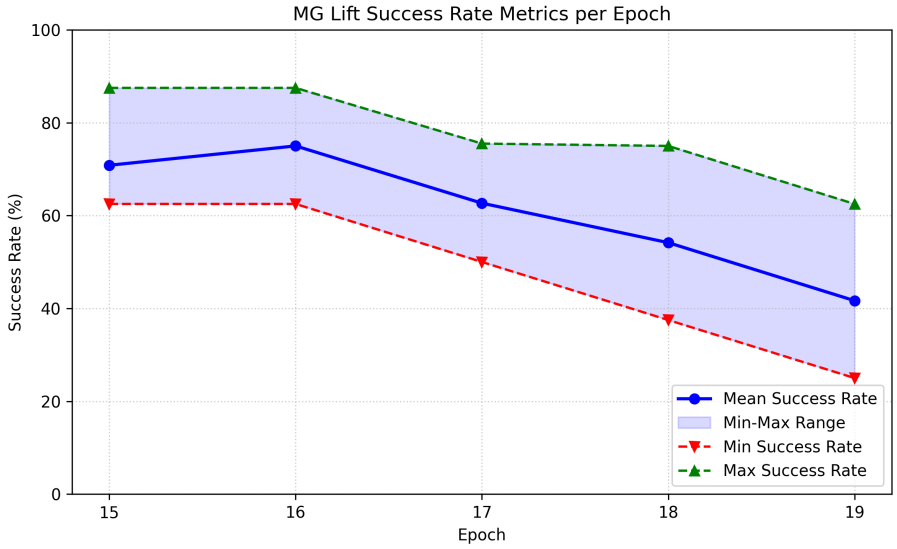


Figure 11: Performance of the model trained with in-context prompts and time based rewards on the block lifting task (MG dataset).

Instead of relying on pure in-context learning we tried adding a small sample of the block stacking demonstrations to the training data. Their representation in the dataset was small (10 block stacking vs 200 block lifting), however we attempted to balance it with weighted sampling. This resulted in improved performance on the main lifting task, and a glimpse of generalization with nonzero, $\sim 10\%$, stacking success rate (see Figure 12).

We see it as an interesting achievement, because the 10 demonstrations alone weren't enough to train a working model, but mixing them into demonstrations of a much easier task sufficed. This might suggest that there exists a possibility for bootstrapping - 10 manually collected demonstrations of a new task are enough for the model to incrementally produce its new training data.

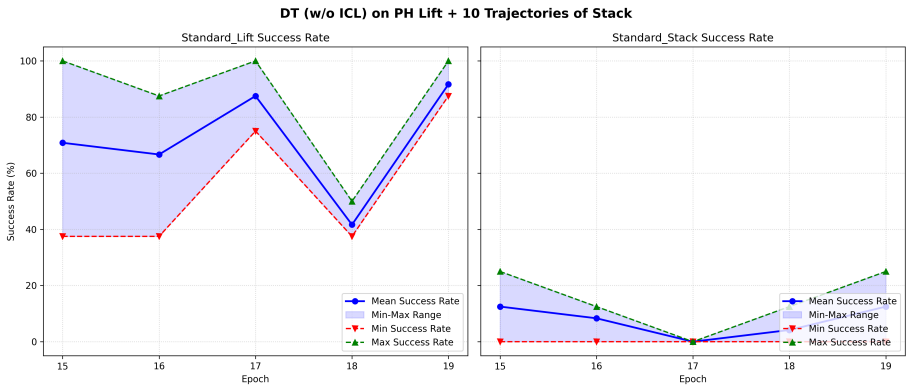


Figure 12: Performance of the model trained on the block lifting (PH) and block stacking tasks.

5 Cross-embodiment Generalization

In this section, we outline the experiments conducted to evaluate the cross-embodiment generalization capabilities of our model. Our primary objective was to enable the model to perform manipulation tasks (such as Lift) using the IIWA arm, for which we have only a few demonstrations, by leveraging the larger dataset of Panda arm demonstrations.

5.1 Baselines and ICL

We began by evaluating the zero-shot performance of models trained on the Panda arm Lift task. Out of the box, these models achieved a 0% success rate, even when using in-context learning (ICL). We found that the model was unsurprisingly unable to learn when trained solely on the 10 available IIWA demonstrations.

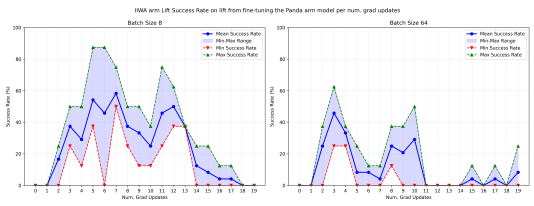
5.2 Test-time adaptation

Next, we evaluated the test-time adaptation capabilities of our model. To do so, we took a model pre-trained on the Panda arm Lift task and fine-tuned it on the 10 IIWA Lift demonstrations. The model achieved a success rate of approximately 50% after only 5 gradient updates with a batch size of 8 (see Figure 13a). Increasing either the number of gradient updates or the batch size did not yield any improvements, and in fact degraded performance (which can be explained by the model overfitting to the small number of available demonstrations).

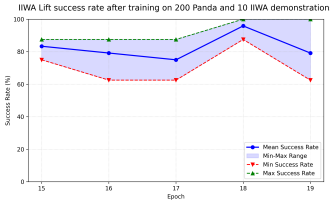
5.3 Training from scratch

To improve these results, we trained a model from scratch on a combined dataset of 10 IIWA and 200 proficient human (PH) Panda Lift demonstrations. To ensure that the IIWA demonstrations were not drowned out by the larger number of Panda demonstrations, we employed weighted sampling. This approach achieved a success rate of approximately 80% - significantly higher and more stable than the test-time adaptation approach (see Figure 13b). This success can be attributed to the fact that mixing in a large number of Panda demonstrations prevented the model from overfitting to the IIWA demonstrations, while the weighted sampling gave the IIWA demonstrations sufficient importance to learn the new embodiment. We emphasize that this is a significant result: it demonstrates that our model is capable of cross-embodiment generalization, thereby achieving performance far superior to what could be obtained using only the limited IIWA demonstrations.

We also attempted to train the model to perform the Stack task with the IIWA arm, but were unable to achieve success. Specifically, we trained it on a combined dataset of Panda Lift, 10 Panda Stack, and 10 IIWA Stack demonstrations, but met with no success. Simultaneously learning a new task and a new embodiment proved to be too challenging for the model.



(a) Test-time adaptation (finetuning)



(b) Training from scratch

Figure 13: Success rates on the IIWA Lift task under different training methodologies.

6 Conclusions